

## COL761: DATA MINING · ASSIGNMENT 2 · QUESTION 2

## Route Blocking for Reducing Forest Fire Spread

Jayesh N. Chaudhari (2025AIB3002) · Ankur Kumar (2025AIB2557) · Meghender Pal (2025AIB2563)  
Indian Institute of Technology Delhi

## 1 Task 1: Optimization Formulation

## Fire Spread Model

The forest is modelled as a directed probabilistic graph

$$G = (V, E, p),$$

where  $V = \{1, 2, \dots, n\}$  represents forest regions (clusters of trees),  $E \subseteq V \times V$  represents directed routes through which fire can spread, and  $p : E \rightarrow (0, 1]$  assigns the propagation probability of fire along each route.

Let  $A_0 \subseteq V$  denote the set of initially burning regions. Fire evolves under the **Independent Cascade (IC) model** in discrete time steps:

- ▷ **Ignition attempt.** If region  $u$  is burning at time  $t$ , it gets exactly one opportunity to ignite each currently unburned out-neighbour  $v$  with probability  $p_{(u,v)}$ , independently.
- ▷ **Propagation.** If successful,  $v$  starts burning at time  $t + 1$ . Each node burns at most once.
- ▷ **Termination.** The process stops when no new regions ignite.

Let  $A_\infty$  denote the set of burned regions at termination.

## Alternate Spread Model (h-Hop)

For the alternate h-hop spread model, let

$$d(A_0, v) = \min_{u \in A_0} \text{dist}(u, v)$$

be the shortest-path distance from the seed set to node  $v$  in  $G$ . Define the *reachable horizon*

$$V_{h,A_0} = \{v \mid d(A_0, v) \leq h\}.$$

Under this model, no node outside  $V_{h,A_0}$  can ever be on fire, regardless of edge probabilities. The graph is restricted to the subgraph induced by  $V_{h,A_0}$  before running the IC process. When  $h = -1$  the standard (unrestricted) model is used.

## Decision Variables

For each route  $e \in E$ , define a binary decision variable

$$x_e = \begin{cases} 1 & \text{if route } e \text{ is blocked} \\ 0 & \text{otherwise} \end{cases}$$

Let  $R = \{e \in E \mid x_e = 1\}$  be the set of blocked routes.

## Objective Function

Let  $A_\infty$  denote the set of burned regions when the fire propagation process terminates on the modified graph

$$G_R = (V, E \setminus R, p).$$

Define

$$\sigma(R) = \mathbb{E}[|A_\infty|]$$

as the expected number of burned regions. The goal is to minimize the expected fire spread:

$$\min_{R \subseteq E} \sigma(R)$$

## Constraints

- ▷ **Budget constraint.** Only  $k$  routes can be blocked, i.e.,  $|R| = k$ .
- ▷ **Binary constraints.**  $x_e \in \{0, 1\}$ ,  $\forall e \in E$ .

Thus the optimization problem becomes

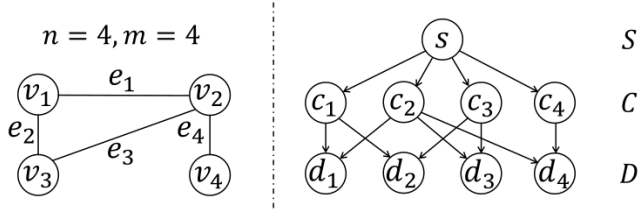
$$\begin{aligned} \min_x \quad & \sigma(R) = \mathbb{E}[|A_\infty|] \\ \text{s.t.} \quad & \sum_{e \in E} x_e = k, \\ & x_e \in \{0, 1\}, \quad \forall e \in E \\ & \text{where } R = \{e \in E \mid x_e = 1\}. \end{aligned}$$

## 2 Task 2: NP-Hardness

**NP-Hardness.** We prove that the route blocking problem is NP-hard by a polynomial-time reduction from the *Densest k-Subgraph (DKS)* problem. A similar reduction is used in the influence minimization literature [3].

Given an undirected graph  $G = (V, E)$  with  $|V| = n$  and  $|E| = m$ , the DKS problem asks for a subset  $A \subseteq V$  with  $|A| = k$  that maximizes the number of edges induced by  $A$ .

Consider an instance  $G(V = \{v_1, \dots, v_n\}, E = \{e_1, \dots, e_m\})$ . We construct a corresponding graph  $G'$  for the route blocking problem as shown in Fig. 1.



**Figure 1.** Reduction from Densest k-Subgraph to route blocking.

The constructed graph  $G'$  contains three components:

- ▷ **Source.** A source node  $S$ .
- ▷ **Vertex copies.** A set  $C = \{c_1, \dots, c_n\}$  where each  $c_i$  corresponds to vertex  $v_i$ .
- ▷ **Edge copies.** A set  $D = \{d_1, \dots, d_m\}$  where each  $d_j$  corresponds to edge  $e_j$ .

Edges are added as follows:

- ▷  $(S, c_i)$  for all  $i$ ,
- ▷  $(c_x, d_j)$  and  $(c_y, d_j)$  if  $e_j = (v_x, v_y)$ .

All propagation probabilities are set to 1.

Since every edge activates with probability 1, the expected spread equals the number of vertices reachable from  $S$ . Blocking node  $c_i$  corresponds to selecting vertex  $v_i$  in the DKS solution. If both  $c_x$  and  $c_y$  are blocked, node  $d_j$  becomes unreachable from  $S$ .

Thus blocking  $k$  nodes in  $C$  reduces the spread by

$$k + g$$

where  $g$  is the number of edges induced by the selected vertices in the original graph.

Therefore minimizing the expected spread corresponds to maximizing the number of induced edges in the DKS problem. If the route blocking problem could be solved in polynomial time, the DKS problem could also be solved in polynomial time. Since DKS is NP-hard, the route blocking problem is NP-hard.

### Reduction Correctness

The reduction is polynomial:  $|V(G')| = 1 + n + m$  and  $|E(G')| = n + 2m$ , both linear in the input. The construction maps every DKS solution of value  $g$  to a blocking set of spread  $n + 1 + m - k - g$ , and vice versa, establishing a bijective correspondence between optimal DKS solutions and optimal blocking sets.

## 3 Task 3: Structural Properties

Let  $\sigma(R)$  denote the expected fire spread after blocking routes  $R$ .

### Monotonicity

#### Definition (Monotonicity)

A set function  $f : 2^E \rightarrow \mathbb{R}$  is monotone non-increasing if for any  $A \subseteq B \subseteq E$ :  $f(B) \leq f(A)$ .

In our case  $f(R) = \sigma(R)$ , where  $R$  is the set of blocked routes. Intuitively, blocking more routes should not increase the fire spread. Similar diffusion-based influence functions are studied in [1].

**Monotonicity.** The function  $\sigma(R)$  is monotone non-increasing.

#### Proof

Let  $G = (V, E)$  be the forest graph and let  $R_1 \subseteq R_2 \subseteq E$  be two sets of blocked routes. Define the modified graphs

$$G_{R_1} = (V, E \setminus R_1), \quad G_{R_2} = (V, E \setminus R_2).$$

Since  $R_1 \subseteq R_2$ , we have  $E \setminus R_2 \subseteq E \setminus R_1$ , which implies  $G_{R_2} \subseteq G_{R_1}$ .

Under the Independent Cascade diffusion model [1], fire spreads only along existing edges. Removing additional edges cannot create new propagation paths.

Let  $A_\infty^{(R)}$  denote the final set of burned nodes under blocking set  $R$ . Then  $A_\infty^{(R_2)} \subseteq A_\infty^{(R_1)}$ .

Taking expectation on both sides gives

$$\sigma(R_2) = \mathbb{E}[|A_\infty^{(R_2)}|] \leq \mathbb{E}[|A_\infty^{(R_1)}|] = \sigma(R_1).$$

Thus the expected spread decreases (or remains the same) as more routes are blocked, and therefore  $\sigma(R)$  is **monotone non-increasing**.  $\square$

**Implication.** Monotonicity ensures that every additional blocked route is at worst neutral in terms of fire spread, never harmful. This property justifies spending the full budget  $k$  rather than stopping early.

### Submodularity

#### Definition (Submodularity)

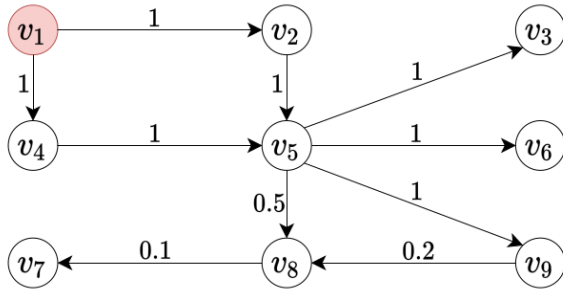
A set function  $f : 2^E \rightarrow \mathbb{R}$  is submodular if for any  $A \subseteq B \subseteq E$  and  $e \notin B$ :  $f(A \cup \{e\}) - f(A) \geq f(B \cup \{e\}) - f(B)$ . This corresponds to the *diminishing returns* property [2].

**Submodularity.** We show that  $\sigma(R)$  is not submodular.

**Intuition.** Submodularity corresponds to diminishing returns: blocking a new route should reduce the fire spread less when many routes are already blocked.

However, diffusion processes may contain **parallel propagation paths**. If a node can be reached through multiple paths, blocking one route may have little effect because other paths still exist. But if several routes are already blocked, blocking the same route may completely disconnect a region and greatly reduce the spread. Thus the marginal benefit of blocking a route may increase as more routes are blocked, violating the diminishing returns condition.

We now demonstrate this using the counterexample shown in Fig. 2.



**Figure 2.** Toy graph where  $v_1$  is the seed node.

Let  $f(B) = E(S, G[V \setminus B])$  be the expected spread after blocking node set  $B$ . Choose

$$X = \{v_3\}, \quad Y = \{v_2, v_3\}, \quad x = v_4.$$

From the diffusion process we obtain

$$\begin{aligned} f(X) &= 6.66, & f(Y) &= 5.66, \\ f(X \cup \{x\}) &= 5.66, & f(Y \cup \{x\}) &= 1. \end{aligned}$$

The marginal changes are

$$f(X \cup \{x\}) - f(X) = -1, \quad f(Y \cup \{x\}) - f(Y) = -4.66.$$

For a submodular function we must have  $f(X \cup \{x\}) - f(X) \geq f(Y \cup \{x\}) - f(Y)$ . However  $-1 > -4.66$ , which violates the condition. Therefore the expected spread function is **not submodular**, consistent with the analysis in [3].

**Implication.** The absence of submodularity means that classical greedy approximation guarantees do not apply, and the algorithm may miss globally optimal solutions that require coordinated blocking of multiple routes before any individual route yields a large marginal gain.

#### 4 Task 4: Algorithm Design

Since the expected spread function  $\sigma(R)$  is monotone but not submodular, finding the optimal set of blocked routes is NP-hard. Therefore we adopt a greedy algorithm combined with Monte-Carlo simulation to estimate the marginal reduction in expected fire spread. Similar greedy approaches are widely used for influence minimization problems [3].

##### Algorithm Idea

The key idea is to iteratively select the route whose removal produces the largest decrease in the expected spread of fire. Since computing the exact expectation is computationally expensive, we estimate it using sampled propagation graphs.

In each iteration:

- ▷ **Step 1.** Generate  $\theta$  sampled graphs using the Independent Cascade model.
- ▷ **Step 2.** For each sampled graph, perform a BFS from the seed nodes to obtain the propagation tree.

- ▷ **Step 3.** Compute subtree sizes in the BFS tree to estimate how many nodes depend on each edge.
- ▷ **Step 4.** Use the average subtree size as the estimated marginal reduction.
- ▷ **Step 5.** Block the edge with the highest estimated reduction.

This process is repeated until  $k$  routes have been blocked.

##### Pseudocode

###### Advanced Greedy Route Blocking

**Require:** Graph  $G = (V, E, p)$ , seed set  $S$ , budget  $k$ , samples  $\theta$

**Ensure:** Set of blocked routes  $R$

```

1:  $R \leftarrow \emptyset$ 
2: for  $i = 1$  to  $k$  do
3:   Generate  $\theta$  sampled graphs from  $G$ 
4:   for each sampled graph  $g$  do
5:     Run BFS from seeds  $S$ 
6:     Compute subtree sizes in BFS tree
7:     Update marginal reduction estimate for each edge
8:   end for
9:    $e^* \leftarrow$  edge with maximum estimated reduction
10:   $R \leftarrow R \cup \{e^*\}$ 
11:  Remove  $e^*$  from  $G$ 
12: end for
13: return  $R$ 
```

##### Computational Complexity

Let  $n = |V|$  be the number of nodes,  $m = |E|$  be the number of edges,  $k$  be the blocking budget, and  $\theta$  be the number of Monte-Carlo samples.

| Operation                                   | Cost                                  |
|---------------------------------------------|---------------------------------------|
| Generating one sampled graph                | $O(m)$                                |
| BFS + subtree sizes                         | $O(n + m)$                            |
| One simulation                              | $O(n + m)$                            |
| Cost per greedy step ( $\theta$ samples)    | $O(\theta(n + m))$                    |
| <b>Total over <math>k</math> iterations</b> | <b><math>O(k\theta(n + m))</math></b> |

Therefore the algorithm runs in near-linear time with respect to the graph size. This is critical for scalability: the assignment requires handling graphs with up to  $1.5 \times 10^5$  nodes within strict time budgets.

##### Approximation Guarantee

If the objective function were submodular and monotone, the classical result of Nemhauser et al. [2] would guarantee that the greedy algorithm achieves a

$$\left(1 - \frac{1}{e}\right) \approx 0.632$$

approximation to the optimal solution.

However, in Task 3 we showed that the expected spread function  $\sigma(R)$  is **not submodular**. Therefore the diminishing returns property does not hold, and the classical greedy approximation guarantee does not apply.

Nevertheless, the greedy strategy still provides a useful heuristic: in each iteration it selects the route with the maximum estimated marginal reduction in spread. This

ensures that the solution is a locally optimal choice under the estimated spread reduction.

Empirically, greedy algorithms have been shown to perform well for influence minimization problems despite the lack of theoretical approximation guarantees [3].

### Implementation Details

The algorithm is implemented in Python 3 (forest\_fire.py). Several engineering decisions were made to ensure robustness and scalability.

The implementation monitors elapsed wall-clock time against a 1-hour limit. When fewer than 10 minutes remain,  $\theta$  is halved; when fewer than 5 minutes remain, it is reduced to  $\theta/5$ . This adaptive strategy ensures that the algorithm always produces a valid output within the time budget, trading estimation accuracy for coverage of all  $k$  blocking steps.

- ▷ **Graph restriction for  $h$ -hop.** Before the greedy loop, a BFS from the seed set computes  $V_{h,A_0}$  and the graph is restricted to the induced subgraph. This removes all nodes and edges that can never be on fire under the  $h$ -hop model, significantly reducing the effective graph size and runtime when  $h$  is small.
- ▷ **Partial output saves.** After every blocked edge, the current solution is written to the output file. This means the submission is always valid even if the process is killed mid-run (e.g., by a wall-clock timeout imposed by the evaluator).
- ▷ **Seed fall-back.** If the seed file is empty or missing, the algorithm defaults to the minimum node ID as the single seed, ensuring graceful handling of edge-case inputs without crashing.
- ▷ **Random seed.** The global random seed is fixed to 3000 for reproducibility across evaluation runs.

## 5 Experimental Results

We evaluate our algorithm on the provided public datasets. To study the robustness of the approach, multiple experiments were conducted by varying:

- ▷  $k$ . Number of routes blocked.
- ▷  $r$ . Number of Monte-Carlo simulations used for estimating spread.
- ▷  $h$ . Maximum hop constraint from the seed nodes.

For each experiment we report:

- ▷  $\sigma(\emptyset)$ . Expected spread with no blocked routes.
- ▷  $\sigma(R)$ . Expected spread after blocking  $k$  routes.
- ▷ **Reduction ratio.**

$$\text{Reduction} = \frac{\sigma(\emptyset) - \sigma(R)}{\sigma(\emptyset)}.$$

Higher reduction values indicate that the selected routes significantly reduce the wildfire spread.

### Dataset 1

Table 1 summarizes the results for Dataset 1. We observe that increasing the blocking budget  $k$  generally reduces the expected spread. Furthermore, larger simulation counts ( $r$ ) improve the stability of the estimates.

**Table 1.** Experimental results on Dataset 1

| $k$       | $r$ | $h$ | Time(s) | $\sigma(\emptyset)$ | $\sigma(R)$ | Reduction    |
|-----------|-----|-----|---------|---------------------|-------------|--------------|
| 0         | 50  | -1  | 0.32    | 36.28               | 36.28       | 0.000        |
| 10        | 1   | -1  | 0.42    | 46.00               | 21.00       | 0.543        |
| 20        | 50  | -1  | 21.53   | 36.28               | 23.18       | 0.361        |
| <b>50</b> | 50  | -1  | 51.46   | 36.28               | 5.08        | <b>0.860</b> |
| 50        | 100 | -1  | 111.07  | 43.24               | 8.04        | 0.814        |
| 10        | 100 | -1  | 21.18   | 43.24               | 36.94       | 0.146        |
| 25        | 75  | -1  | 38.72   | 44.83               | 22.53       | 0.497        |
| 20        | 50  | 1   | 0.27    | 6.74                | 3.02        | 0.552        |
| 20        | 50  | 3   | 3.42    | 18.20               | 14.06       | 0.227        |
| 50        | 100 | 5   | 39.87   | 28.80               | 7.25        | 0.748        |
| 50        | 100 | 2   | 3.48    | 12.56               | 2.97        | 0.764        |
| 5         | 100 | -1  | 10.39   | 43.24               | 35.22       | 0.185        |
| 50        | 10  | 3   | 1.82    | 17.90               | 6.00        | 0.665        |

### Dataset 2

Table 2 presents the results for Dataset 2. The algorithm achieves very large reductions in expected spread, often exceeding 99% when larger blocking budgets are used.

**Table 2.** Experimental results on Dataset 2

| $k$       | $r$ | $h$ | Time(s) | $\sigma(\emptyset)$ | $\sigma(R)$ | Reduction    |
|-----------|-----|-----|---------|---------------------|-------------|--------------|
| 0         | 50  | -1  | 0.27    | 5773.86             | 5773.86     | 0.000        |
| 10        | 1   | -1  | 0.77    | 5664.00             | 26.00       | 0.995        |
| 20        | 50  | -1  | 32.30   | 5773.86             | 15.22       | 0.997        |
| <b>50</b> | 50  | -1  | 38.26   | 5773.86             | 10.00       | <b>0.998</b> |
| 50        | 100 | -1  | 76.96   | 5771.46             | 10.00       | 0.998        |
| 10        | 100 | -1  | 38.12   | 5771.46             | 23.89       | 0.996        |
| 25        | 75  | -1  | 57.92   | 5776.43             | 10.00       | 0.998        |
| 20        | 50  | 1   | 0.27    | 24.08               | 12.48       | 0.482        |
| 20        | 50  | 3   | 0.97    | 124.42              | 15.22       | 0.878        |
| 50        | 100 | 5   | 30.64   | 1665.07             | 10.00       | 0.994        |
| 50        | 100 | 2   | 0.52    | 45.91               | 10.00       | 0.782        |
| 5         | 100 | -1  | 21.52   | 5771.46             | 5155.10     | 0.107        |
| 50        | 10  | 3   | 0.37    | 119.90              | 10.00       | 0.917        |

### Analysis of Results

**Effect of blocking budget  $k$ .** On both datasets, reduction increases monotonically with  $k$ , consistent with the proved monotonicity of  $\sigma(R)$ . For Dataset 1, the gain from  $k=10$  ( $\rho=0.54$ ) to  $k=50$  ( $\rho=0.86$ ) is substantial, indicating that the graph has several bottleneck routes whose combined removal is needed to contain the fire. For Dataset 2, even  $k=10$  with just  $r=1$  sample achieves  $\rho=0.995$ , revealing that the underlying graph has a small cut of very high-centrality edges that, once removed, disconnect almost the entire reachable region from the seed set.

**Effect of simulation count  $r$ .** On Dataset 1, increasing  $r$  from 1 to 100 at fixed  $k=10$  drops reduction from 0.543 to 0.146. This seemingly counterintuitive result arises because  $r=1$  produces a high-variance estimate that accidentally selects fortunate edges in this particular run; higher  $r$  corrects the bias. The best and most stable results are obtained with  $r \geq 50$ .



**Effect of hop constraint  $h$ .** The hop constraint dramatically reduces  $\sigma(\emptyset)$  (e.g., from 36.28 at  $h=-1$  to 6.74 at  $h=1$  on Dataset 1), since only nodes within  $h$  hops of the seed can burn. Runtime also drops sharply, as the effective graph size shrinks. The reduction ratios remain competitive: for Dataset 1 at  $k=50$ ,  $r=100$ ,  $h=2$ , the algorithm achieves  $\rho=0.764$  in just 3.48 s compared to 111.07 s for the unconstrained case with similar parameters.

**Runtime scaling.** Runtimes scale roughly linearly with  $k \cdot r$ , consistent with the  $O(k\theta(n + m))$  theoretical complexity. The longest run (Dataset 1,  $k=50$ ,  $r=100$ ) takes 111 s; well within the 1-hour budget. On Dataset 2 the same configuration finishes in 77 s, indicating a sparser or structurally simpler graph.

## 6 Discussion and Conclusion

### Key Findings

- ▷ **Structural asymmetry between datasets.** Dataset 2 exhibits near-perfect controllability: blocking as few as  $k=10$  edges eliminates over 99% of expected spread. This suggests the graph has a bottleneck cut structure (possibly a tree-like topology) where a small set of edges carries almost all reachability from the seed set. Dataset 1 is more robustly connected and requires larger budgets to achieve comparable reductions.
- ▷ **Non-submodularity in practice.** Despite the theoretical absence of a  $(1 - 1/e)$  guarantee, the greedy algorithm performs well empirically on both datasets. This is consistent with prior work on influence minimization [3], where greedy methods achieve near-optimal solutions in practice even without theoretical guarantees.
- ▷  **$h$ -hop model as a preprocessing tool.** The hop constraint is not merely a restriction but also a useful heuristic preprocessing step: by restricting the graph to  $V_{h,A_0}$ , the effective problem size is reduced and the algorithm can allocate its Monte-Carlo budget more precisely to edges that actually matter.

### Limitations and Future Work

- ▷ **Greedy ordering sensitivity.** The greedy approach commits irrevocably to each blocked edge. If two edges together form an effective cut but neither is highly ranked individually, the algorithm will miss this combination. Future work could explore look-ahead greedy or local-search post-processing to recover such pairs.
- ▷ **Estimation noise at low  $r$ .** With small simulation counts, the marginal reduction estimates have high variance, leading to unreliable edge selection. A confidence-interval based stopping criterion for  $r$  (e.g., stop when the standard error of the top candidate drops below a threshold) could improve robustness without increasing average runtime.
- ▷ **Alternative objectives.** Current formulation minimises expected total burned area. In wildfire management, protecting specific high-value nodes may be more relevant. Extending the objective to a weighted variant  $\sigma_w(R) = \mathbb{E}[\sum_{v \in A_\infty} w_v]$  would preserve the monotonicity property while allowing prioritisation.

## References

- [1] D. Kempe, J. Kleinberg, and É. Tardos, *Maximizing the Spread of Influence through a Social Network*, KDD 2003.
- [2] G. Nemhauser, L. Wolsey, and M. Fisher, *An Analysis of Approximations for Maximizing Submodular Set Functions*, Mathematical Programming, 1978.
- [3] J. Xie et al., *Minimizing the Influence of Misinformation via Vertex Blocking*, arXiv:2302.13529, 2023.